QUY ƯỚC LÀM VIỆC NHÓM & PHÂN CÔNG ĐỒ ÁN

Dự án: Triển khai và Đánh giá So sánh Hệ thống MLOps Đa Nền tảng (Arch Linux vs. Ubuntu Server)

Môn học: Linux và Phần mềm Mã nguồn mở

Nhóm: 3 người

Tech Stack: Bash (IaC), Docker / Rootless Docker, Docker Compose, MLflow, PostgreSQL (Metadata Store), MinIO (Artifact Store), FastAPI (Model Serving), Prometheus + Grafana (Monitoring), nftables (Arch), UFW (Ubuntu).

---

1. PHÂN CÔNG NHIỆM VỤ (3 THÀNH VIÊN)

🧑‍💻 Thành viên 1: Toàn quy n Node Arch Linux (Rolling Release & Hardening)

• **Trọng tâm:** Triển khai độc lập 100% Node Arch Linux, bảo mật hệ thống sâu (The Arch Way), Rootless Docker và MLOps Stack.

• **Chi tiết nhiệm vụ:**

1. **OS Hardening & Security (Arch Base):**
   • Thiết lập user độc lập `mlops` (tách biệt tài khoản cá nhân).
   • Hardening SSH Daemon: Port `2222`, cấm `root` login, giới hạn số lần xác thực.
   • Tường lửa **`nftables`** stateful: Tích hợp rate-limiting chống brute-force SSH (tối đa 4 kết nối/phút), drop invalid packets, mở các port public (`2222`, `8000`, `5000`, `3000`, `9001`).
   • Tinh chỉnh Linux Kernel qua `sysctl` (`fs.file-max`, `net.core.somaxconn`, `net.ipv4.tcp_tw_reuse`).
   • Viết script cài đặt (`01_system_hardening.sh`) và rollback an toàn (`01_rollback_hardening.sh`).
2. **Rootless Docker Engine:**
   • Cấu hình User Namespaces (`/etc/subuid`, `/etc/subgid` với ánh xạ `mlops:100000:65536`).
   • Cấu hình cgroups v2 resource delegation cho unprivileged user.
   • Kích hoạt `loginctl enable-linger mlops` để container chạy ngầm khi logout.
3. **MLOps Core Stack (Docker Compose):**
   • Thiết kế `docker-compose.yml` phân tách 2 mạng: `mlops-backend` (cô lập Postgres 5432, MinIO API 9000) và `mlops-frontend`.
   • FastAPI Model Serving: Multi-stage build, non-root user (`USER 10001`), endpoint `/v1/predict`, `/v1/models/reload`, xuất metrics `/metrics`.
4. **Systemd Lifecycle:**
   • Viết User Systemd Service (`mlops-stack.service`) tự động khởi động stack khi boot.

🧑‍💻 Thành viên 2: Node Ubuntu — H t ng & B o m t (Infrastructure & Security)

• **Trọng tâm:** Triển khai nền tảng hạ tầng, bảo mật hệ điều hành Ubuntu Server (22.04/24.04 LTS), cài đặt Docker chuẩn Debian và quản trị dịch vụ hệ thống.

• **Chi tiết nhiệm vụ:**

1. **OS Hardening & Security (Ubuntu Base):**
   • Thiết lập user deploy `mlops`.
   • Hardening SSH Daemon: Đổi port `2222`, cấm đăng nhập `root`.
   • Tường lửa **`ufw`** chuẩn Ubuntu: Cấu hình mặc định deny incoming, mở đúng các port quy định (`2222`, `8000`, `5000`, `3000`, `9001`).
   • Viết script cài đặt (`ubuntu/scripts/01_system_hardening.sh`) và rollback (`01_rollback_hardening.sh`).
2. **Docker Engine & Daemon Management:**
   • Script tự động cài đặt Docker Engine và Docker Compose plugin chính thức qua kho APT (`download.docker.com`).
   • Cấu hình Docker daemon cấp hệ thống qua systemd root (`systemctl enable --now docker`).
   • Cấu hình log rotation cho Docker daemon để tránh đầy ổ cứng host.
3. **Chạy & Kiểm thử Stack trên Ubuntu:**
   • Chạy toàn bộ stack Docker Compose trên Ubuntu Server.
   • Kiểm tra tính toàn vẹn của mạng nội bộ và các endpoint dịch vụ trên Ubuntu.

🧑‍💻 Thành viên 3: Node Ubuntu — Observability, ML Pipeline & Benchmarking

• **Trọng tâm:** Giám sát hệ thống (Prometheus + Grafana), tự động hóa huấn luyện ML Pipeline và thực hiện kịch bản đo lường Benchmark so sánh giữa 2 hệ điều hành.
• **Chi tiết nhiệm vụ:**

1. **Hệ thống Giám sát (Observability):**
   • Cấu hình Prometheus (`prometheus.yml`): Cào metrics từ FastAPI và Node Exporter.
   • Thiết lập cảnh báo Prometheus Alert Rules (cảnh báo khi API độ trễ cao hoặc service down).
   • Provisioning Grafana Dashboard: Tự động nạp DataSource và thiết kế Dashboard theo dõi tài nguyên (CPU, RAM, Disk I/O) và hiệu năng API (RPS, Latency p95/p99).
2. **ML Pipeline Automation:**
   • Viết kịch bản tự động hóa (`train_and_register.py`): Fetch dữ liệu, train model Scikit-learn.
   • Ghi log siêu tham số, accuracy vào PostgreSQL thông qua MLflow Tracking Server.
   • Tự động upload artifact model `.pkl` lên MinIO S3 bucket và gắn nhãn Model Registry là `Production`.
3. **Benchmarking & Báo cáo so sánh đối kháng:**
   • Viết script load test tự động (`benchmark.sh`): Đo RPS và Latency của FastAPI khi chịu tải trên Arch Linux (Rootless Docker) so với Ubuntu Server (Root Docker).
   • Đo lường tiêu thụ RAM tĩnh (idle sau khi boot) và RAM động (khi full stack hoạt động).
   • Tổng hợp bảng số liệu so sánh, vẽ biểu đồ đối sánh đưa vào slide thuyết trình và báo cáo đồ án.

---

2. QUY ƯỚC GIT

2.1. Phân nhánh (Branching Strategy)

- `main`: Nhánh nộp bài / Production – **KHÔNG push trực tiếp**.
- `dev`: Nhánh tích hợp và test chung – **KHÔNG push trực tiếp**.
- `feat/arch-*`: Toàn bộ tính năng thuộc Node Arch Linux (Thành viên 1).
- `feat/ubuntu-infra-*`: Tính năng hạ tầng, hardening, docker trên Ubuntu (Thành viên 2).
- `feat/ubuntu-mlops-*`: Tính năng monitoring, pipeline, benchmark trên Ubuntu (Thành viên 3).
- `fix/*`: Sửa lỗi hệ thống.
- `docs/*`: Cập nhật tài liệu, slide và báo cáo.

2.2. Chuẩn Commit Message (Conventional Commits)

- `feat(arch)`: Script và cấu hình cho Arch Linux.
- `feat(ubuntu-infra)`: Script hardening, ufw, cài đặt Docker trên Ubuntu.
- `feat(ubuntu-mlops)`: Script pipeline, prometheus, grafana, benchmark.
- `fix(security)`: Sửa lỗi bảo mật, tường lửa, SSH.
- `docs(report)`: Cập nhật tài liệu báo cáo và số liệu đối sánh.

Ví dụ:

```bash
feat(arch): configure rootless docker namespaces and lingering
feat(ubuntu-infra): implement automated ufw rules and unattended docker install
feat(ubuntu-mlops): add train_and_register script with mlflow s3 integration
feat(ubuntu-mlops): add latency and rps benchmark script
docs(report): update comparison table and system architecture diagram
```

2.3. Quy tắc làm việc trên Git

- **Quy trình Pull Request (PR):**

1. Tạo nhánh từ `dev`: `git checkout dev && git pull && git checkout -b feat/ten-nhanh`
2. Viết code, kiểm tra chạy thử đạt yêu cầu.
3. Push lên remote: `git push origin feat/ten-nhanh`
4. Mở PR vào `dev` (mỗi PR không quá 500 dòng code).
5. Có ít nhất 1 thành viên khác review và Approve -> Merge.
6. Xóa branch sau khi merge hoàn tất.

- **Quy tắc an toàn:**
- **Tuyệt đối không commit file `.env` chứa mật khẩu thật lên Git.**
- Không dùng `git push --force` lên nhánh `dev` và `main`.
- Luôn pull nhánh `dev` về máy vào đầu mỗi phiên làm việc.

---

3. TIÊU CHUẨN KỸ THUẬT & CODE STYLE

3.1. Bash Script (Infrastructure as Code)

• Luôn khai báo `set -euo pipefail` ở đầu mọi script để bắt lỗi lập tức.

• Có trap hiển thị dòng code và câu lệnh lỗi (`trap '...' ERR`).

• Đảm bảo tính **Idempotent**: Chạy lại nhiều lần không sinh lỗi hoặc ghi trùng config.

• Bắt buộc có script **Rollback** tương ứng (`01_rollback_*.sh`) cho mọi script thay đổi hệ thống.

• Không viết comment dư thừa trong script; ưu tiên đặt tên biến rõ nghĩa và in log màu sắc (`log_info`, `log_success`, `log_error`).

3.2. Container & Kiến trúc MLOps

• **Kiến trúc Zero Trust:**

• Cổng 5432 (PostgreSQL) và Cổng 9000 (MinIO S3 API) nằm trọn trong mạng nội bộ `mlops-backend`. Host không bind cổng ra `0.0.0.0`.

• Chỉ các cổng UI/API cần giao tiếp ngoài mới bind ra host: `8000` (FastAPI), `5000` (MLflow UI), `3000` (Grafana), `9001` (MinIO Web Console).

• **Multi-Stage Build:** Dockerfile FastAPI dùng multi-stage build để tối ưu dung lượng và chạy bằng tài khoản unprivileged (`USER 10001`).

---

4. QUẢN LÝ BIẾN MÔI TRƯỜNG (.env.example)

Chỉ commit file mẫu `.env.example`:

```bash
# PostgreSQL (Metadata Store cho MLflow)
POSTGRES_USER=mlops
POSTGRES_PASSWORD=secure_postgres_password_change_me
POSTGRES_DB=mlflow_db
POSTGRES_PORT=5432
# MinIO (Artifact S3 Object Store cho MLflow)
MINIO_ROOT_USER=minioadmin
MINIO_ROOT_PASSWORD=secure_minio_password_change_me
MINIO_API_PORT=9000
MINIO_CONSOLE_PORT=9001
MLFLOW_BUCKET_NAME=mlflow-artifacts
# MLflow Tracking Server UI
MLFLOW_PORT=5000
# FastAPI Serving
FASTAPI_PORT=8000
MODEL_NAME=ProductionModel
# Telemetry
PROMETHEUS_PORT=9090
GRAFANA_PORT=3000
GRAFANA_ADMIN_PASSWORD=admin
```

```
NODE_EXPORTER_PORT=9100
```